

Prime Performance Evaluation

Report 4

Source Code and GitHub Repositories

The files used for this evaluation can be found at the following repository:

<https://github.com/sam444555/cs2910>

The original Prime code used is available at the following Johns Hopkins DSN repository:

<https://github.com/jhu-dsn/prime>

Prime Pre-Order Flow Control

- **MAX_PO_IN_FLIGHT** (*prime/src/def.h*) describes the maximum number of outstanding pre-order requests allowed at a time before their execution
- Once reached, Prime detaches the IPC client socket (*prime/src/pre_order.c*) in the function **PRE_ORDER_Send_PO_Request()**
- When the number of pending pre-order requests falls below this threshold, Prime reattaches the IPC client socket (*prime/src/order.c*) in the function **ORDER_Execute_Commit()**

Issues with the Current Implementation (periodic sending disabled)

- Update passes through Prime's pre-order message processing pipeline:
 - (1) Message received at **Net_Srv_Recv()** (*network.c*)
 - (2) Gets passed to **PROCESS_Message()** (*process.c*) to determine message type
 - (3) Message is identified as an UPDATE and gets forwarded to **PRE_ORDER_Process_Update()** (*pre_order.c*) and adds the update, if valid, to the outgoing Pre-Order request queue **DATA.PO.po_request_dll**
 - (4) If periodic sending is disabled, Prime instantly tries to send this message by invoking **PRE_ORDER_Send_PO_Request()** *pre_order.c*
 - Case 1: **MAX_PO_IN_FLIGHT threshold has been reached**
 - Socket gets detached, request remains in queue & function returns
 - Case 2: **Threshold not yet reached**
 - Update gets turned into a po_request by calling **PRE_ORDER_Construct_PO_Request()** (*packets.c*) and eventually winds up in **DATA.PO.History[i]** (where *i* is the replica's id)

○ ISSUE

- When Case 1 is true, the update does not get turned into a request and remains in **DATA.PO.po_request_dll**
- New messages received by Prime (once socket gets reattached) in **Net_Srv_Recv()** should reinitiate this pipeline and drain the list
- If all updates have been received & Prime detaches the client socket described in Case 1 then this pipeline will never reinitiate & any updates will remain in **DATA.PO.po_request_dll**
 - PRE_ORDER_Construct_PO_Request() only invoked by PRE_ORDER_Send_PO_Request()
 - PRE_ORDER_Send_PO_Request() only invoked by:
 - Periodic PO Sending is Enabled
 - PR_Resume_Normal_Operations() (proactive_recovery.c) – called only during replica startup/recovery
 - Previously discussed message processing pipeline
- When the socket gets reattached in order.c, **Prime resumes listening for new client updates but does not explicitly process updates already waiting in DATA.PO.po_request_dll**
 - With no periodic sending or recovery, the message will sit in queue indefinitely

○ SOLUTION

- **OPTION A:** Enable periodic sending
 - Set **SEND_PO_REQUESTS_PERIODICALLY** to 1 in prime/src/def.h
- **OPTION B:** Fix the bug
 - In order.c function ORDER_Execute_Commit, after Prime has done the following:
 - (1) reattaches the client socket in ORDER_Execute_Commit()
 - (2) Updates the value of the highest executed PO sequence number (DATA.PO.po_seq_executed) in ORDER_Execute_Commit()
 - Process all the updates that remain in the list by invoking PRE_ORDER_Send_PO_Request() only when:

- A previous detachment was detected
 $((DATA.PO.po_seq.seq_num - DATA.PO.po_seq_executed.seq_num) == (MAX_PO_IN_FLIGHT))$
- The list is not empty
- Periodic sending is disabled

Solution: Option B observations

- Freezing behavior eliminated
- Throughput decreases by ~5%
- Avg latency increases by ~5% as well

Single Driver Configuration				
Link Speed	Total Sends	Number of Emulated Clients		MAX_PO_IN_FLIGHT
∞	20k	50		20
Metrics	With Fix		No Fix	Diff (Fix vs No Fix)
Total Time	20.13 secs		19.16 secs	+5.06%
Throughput (Expected)	996.31 updates/sec		1047.96 updates/sec	-4.93%
Throughput (Actual)	993.73 updates/sec		1043.72 updates/sec	-4.79%
Min Latency	16.079 ms		16.634 ms	-3.34%
Max Latency	79.430 ms		76.525 ms	+3.8%
Avg Latency	50.185 ms		47.712 ms	+5.18%
Total Freezes	0/50		2/50	-100%

Performance Testing: Docker & Aster

(ASTER) Single Driver Configuration (Unlimited Mbps link speed, 25k total sends, MAX_PO_IN_FLIGHT=20)							
Avg RTT: 0.0362 ms							
Num Emulated Clients [1,5,10,15,20,25,30,50,100] Test Averages over 5 trials							
			Throughput (updates/sec)		Latency (ms)		
Num Emulated Clients	Elapsed Time (seconds)	Stalled Sends	Expected	Actual	Min	Max	Avg
1	501.336	0	50.04	49.87	16.1504	27.7664	19.9832
5	100.294	0	250.132	249.274	15.6546	35.2998	19.9894
10	50.164	0	500.072	498.36	16.6042	35.0734	19.9968
15	33.468	0	749.706	746.98	15.8242	39.9744	20.0076
20	25.11	0	999.138	995.628	17.3578	39.0912	20.0174
25	25.092	0	999.114	996.218	16.0392	56.0962	25.0224
30	25.11	0	998.252	995.622	14.5452	59.4746	30.0524
50	25.086	0	998.598	996.552	22.1182	77.0042	50.0702
100	25.088	0	998.974	996.554	18.5124	116.906	100.1028

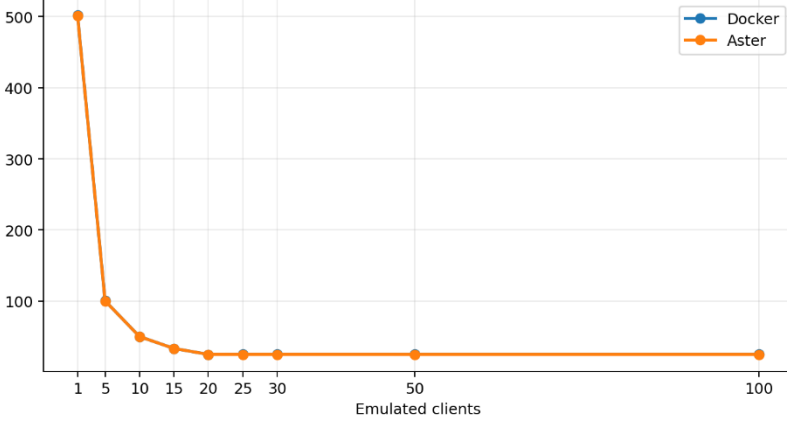
(DOCKER) Single Driver Configuration (Unlimited Mbps link speed, 25k total sends, MAX_PO_IN_FLIGHT=20)							
Num Emulated Clients [1,5,10,15,20,25,30,50,100] Test Averages over 5 trials							
			Throughput (updates/sec)		Latency (ms)		
Num Emulated Clients	Elapsed Time (seconds)	Stalled Sends	Expected	Actual	Min	Max	Avg
1	501.74	0	50.03	49.828	15.006	35.4016	19.989
5	100.402	0	249.882	249.002	15.1032	39.16	20.0092
10	50.218	0	499.614	497.834	15.5986	39.9154	20.0156
15	33.49	0	749.028	746.448	16.9526	36.4356	20.026
20	25.134	10.2	998.236	994.714	17.1194	39.8464	20.0354
25	25.14	2101.8	997.524	994.446	17.1392	50.2096	25.0618
30	25.156	56540.8	996.876	993.836	18.0468	56.9576	30.094
50	25.16	7297750.8	996.38	993.622	23.2822	79.1682	50.1814
100	25.172	27456097.2	996.038	993.102	24.3596	121.1636	100.3978

Aster Network Latency Test — Local Network 1 (192.168.53.x)								
No emulation								
Source: aster13 (192.168.53.22)								
Destination	IP Address	Packets Sent	Packets Received	Packet Loss	Min RTT (ms)	Avg RTT (ms)	Max RTT (ms)	Mdev (ms)
aster14	192.168.53.23	10	10	0%	0.031	0.037	0.066	0.009
aster15	192.168.53.24	10	10	0%	0.03	0.034	0.048	0.005
aster16	192.168.53.25	10	10	0%	0.031	0.035	0.053	0.006
aster17	192.168.53.26	10	10	0%	0.034	0.039	0.054	0.005
aster18	192.168.53.27	10	10	0%	0.03	0.036	0.051	0.005

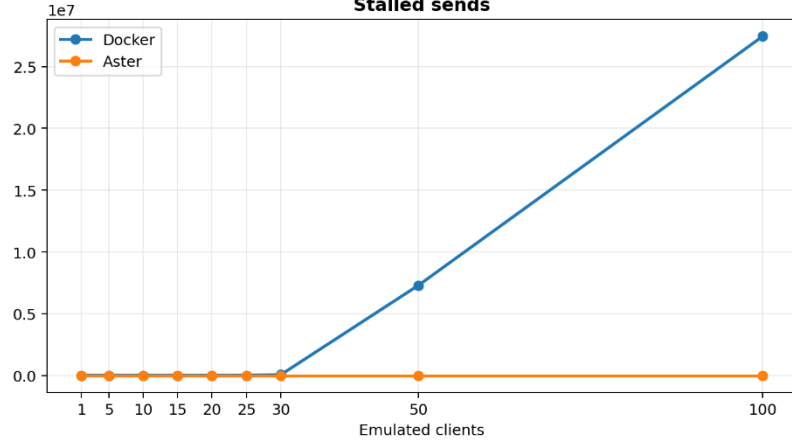
Docker Network Test								
No emulation								
Source: prime1 container								
Destination	IP Address	Packets Sent	Packets Received	Packet Loss	Min RTT (ms)	Avg RTT (ms)	Max RTT (ms)	Mdev (ms)
prime2	172.20.0.3	50	50	0%	0.013	0.024	0.055	0.009
prime3	172.20.0.4	50	50	0%	0.013	0.025	0.057	0.009
prime4	172.20.0.5	50	50	0%	0.012	0.024	0.048	0.008
prime5	172.20.0.6	50	50	0%	0.014	0.026	0.093	0.012
prime6	172.20.0.7	50	50	0%	0.012	0.025	0.086	0.011

Docker vs. Aster — Average Benchmark Comparison

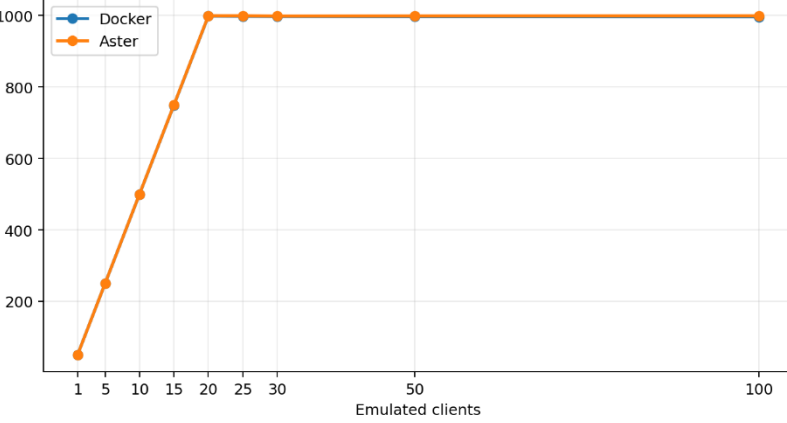
Elapsed time (seconds)



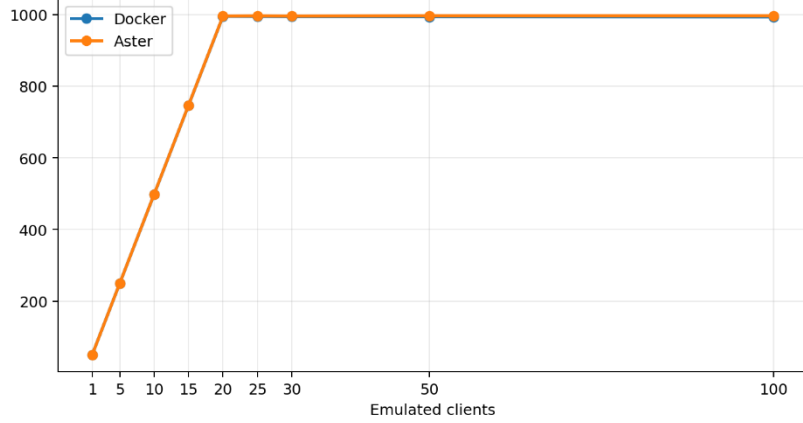
Stalled sends



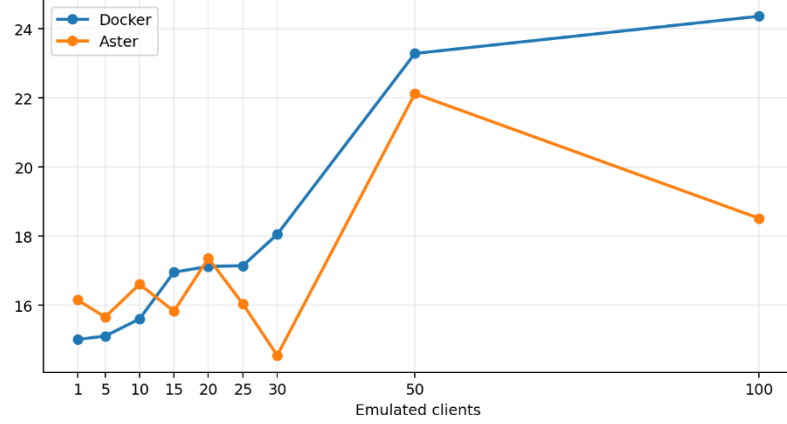
Expected throughput (updates/sec)



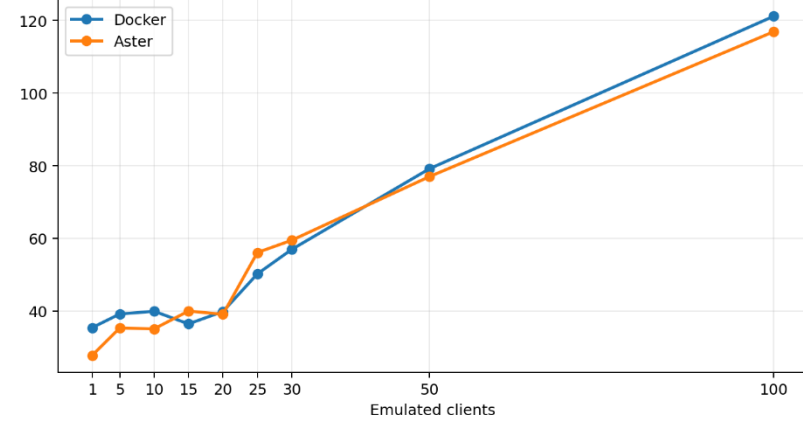
Actual throughput (updates/sec)



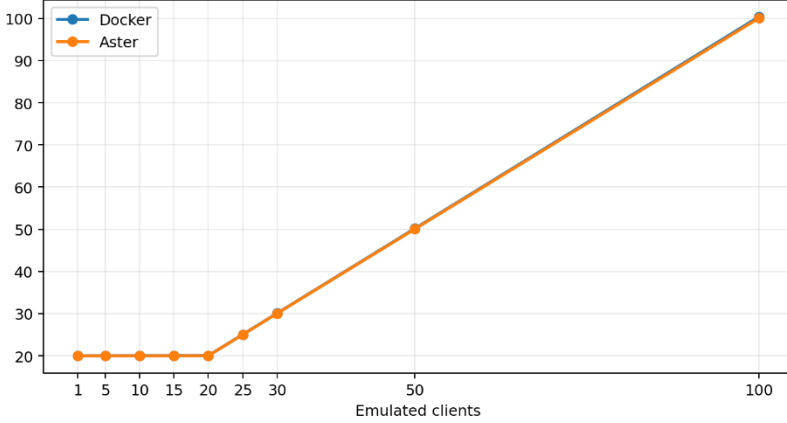
Minimum latency (ms)



Maximum latency (ms)



Average latency (ms)



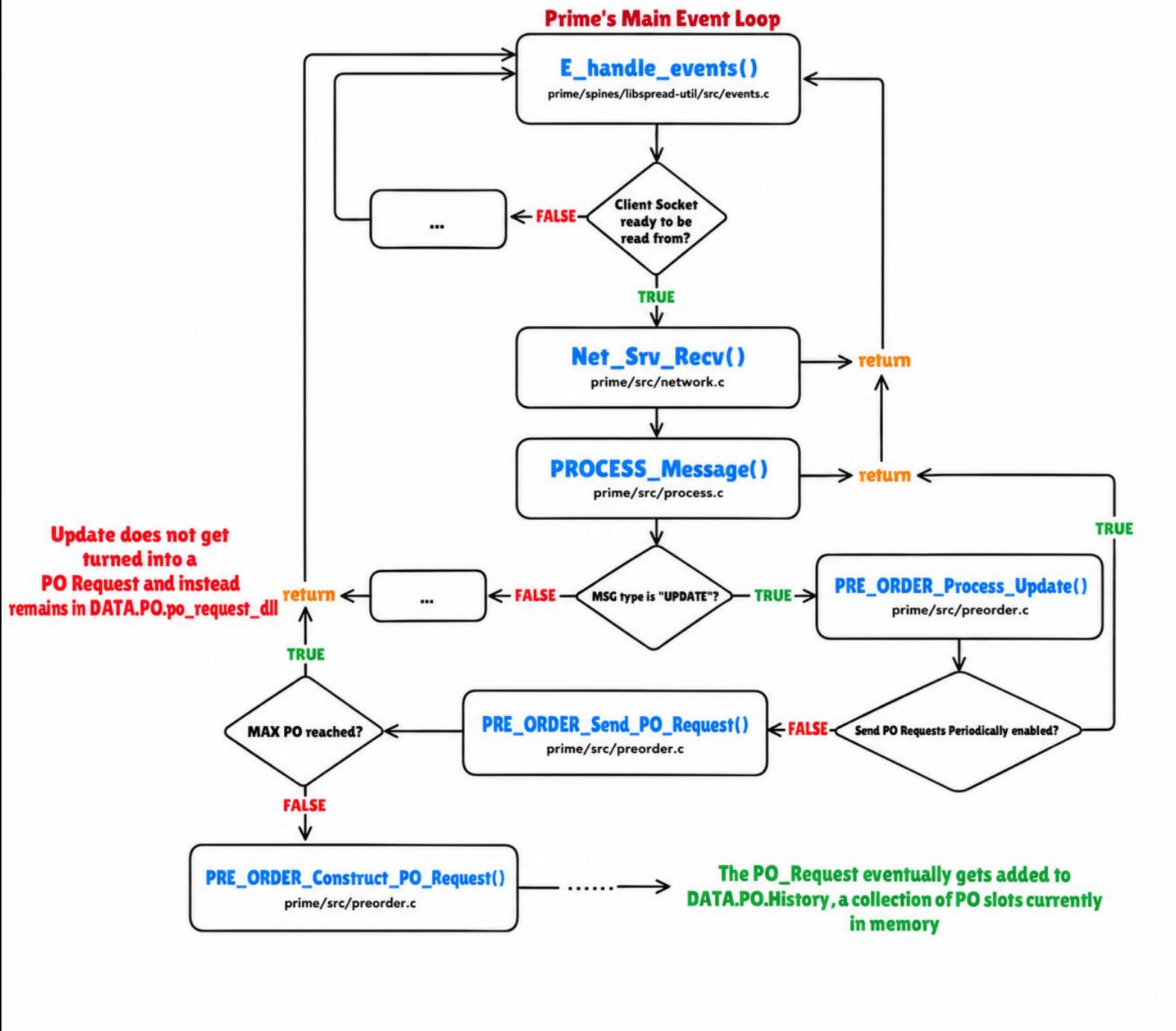


Figure 1: Control flow of a client update through Prime